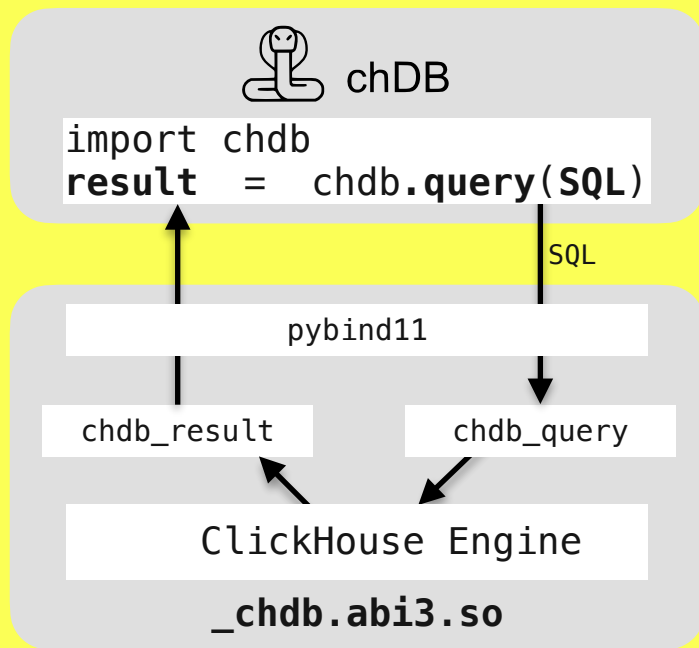


chDB

基于ClickHouse的高性能嵌入式SQL引擎

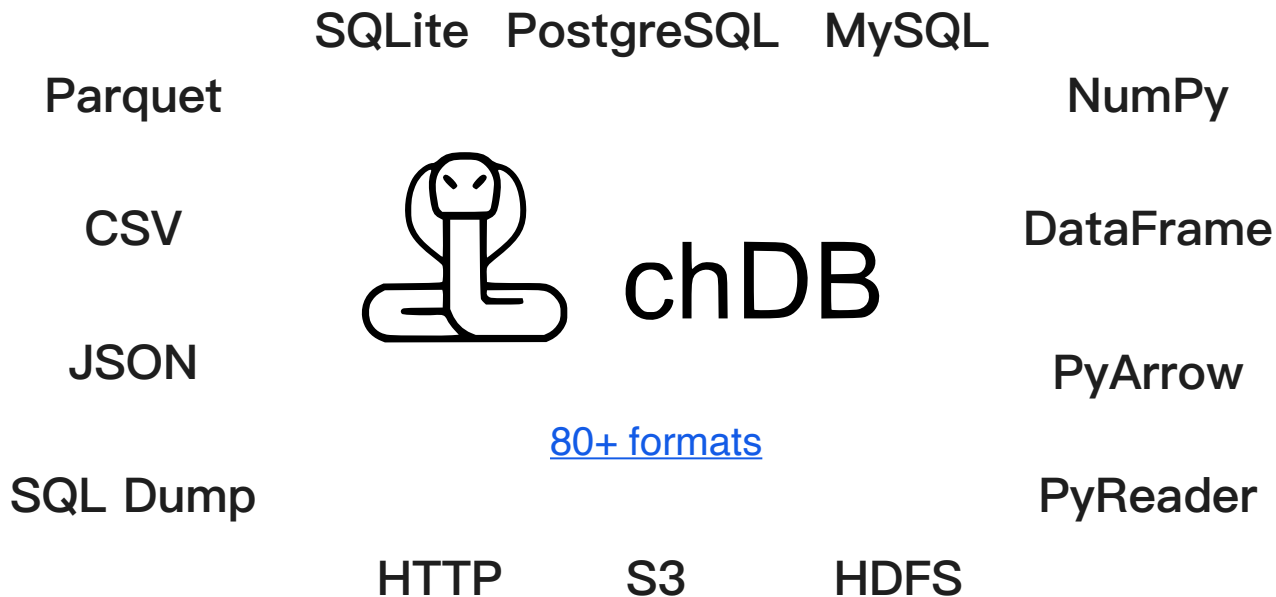
什么是chDB?

- In-process SQL数据分析引擎
- Serverless — 无需安装 / 运行 ClickHouse
- 基于 ClickHouse
- 支持多种语言: Python、Go、Rust、Node.js、Bun、.NET、Ruby
- 深度集成 Python 生态



chDB能帮我做些什么？

万物皆为表



直接查询和输出 Python 对象

```
import chdb
import pandas as pd
import pyarrow as pa

data = {
    'c1': [1, 2, 3],
    'c2': [
        {'number': 4, 'tags': ['python', 'data']},
        {'number': 5, 'tags': ['sql', 'analytics']},
        {'number': 6, 'tags': ['ml', 'ai', 'python']}
    ]
}

df = pd.DataFrame(data)
arrow_table = pa.table(data)

print(chdb.query("SELECT c1, c2.number, c2.tags[2] as tag FROM Python(df)", "DataFrame"))
print(chdb.query("SELECT c1, c2.number, c2.tags[2] as tag FROM Python(arrow_table)", "ArrowTable"))
```

关联多个数据源



chDB

```
chdb.query("""
SELECT name, age, sex, some new tag, ...
FROM url('some_http_data.parquet') big
LEFT JOIN file('some_new_data.csv') local_csv
ON big.uid = local_csv.uid
LEFT JOIN Python(some_processed_df) df
ON df.uid = big.uid
LIMIT 1000
""").show()
```



pandas

你的 Python 专属表引擎

```
import chdb

class myReader(chdb.PyReader):
    def __init__(self, data):
        # do some init

        self.cursor = 0

    def read(self, col_names, count):
        # return block like data[cursor:cursor+count]

        self.cursor += count
        return ret

reader = myReader()

chdb.query("SELECT b, sum(a) FROM Python(reader) GROUP BY b").show()
```

流式查询

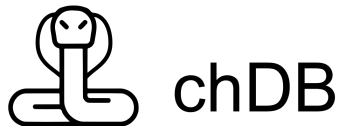


```
import chdb

conn = chdb.connect()
query = "SELECT number FROM numbers(500000)"

# Streaming query: retrieve results by block and process
with conn.send_query(query, "DataFrame") as stream_result:
    for chunk_df in stream_result:
        print(chunk_df)

conn.close()
```

★ chDB 支持 Python/Golang/Rust/Node.js/Bun/.NET/Ruby



```
import chdb
res = chdb.query('select version()', 'CSV');
```



_chdb.abi3.so



```
const chdb = require("chdb-node");
var result = chdb.Execute("SELECT version()", "CSV");
```



package main

```
import (
    "github.com/chdb-io/chdb-go/chdb"
)
```

```
func main() {
    result := chdb.Query("SELECT version()", "CSV")
}
```



Golang

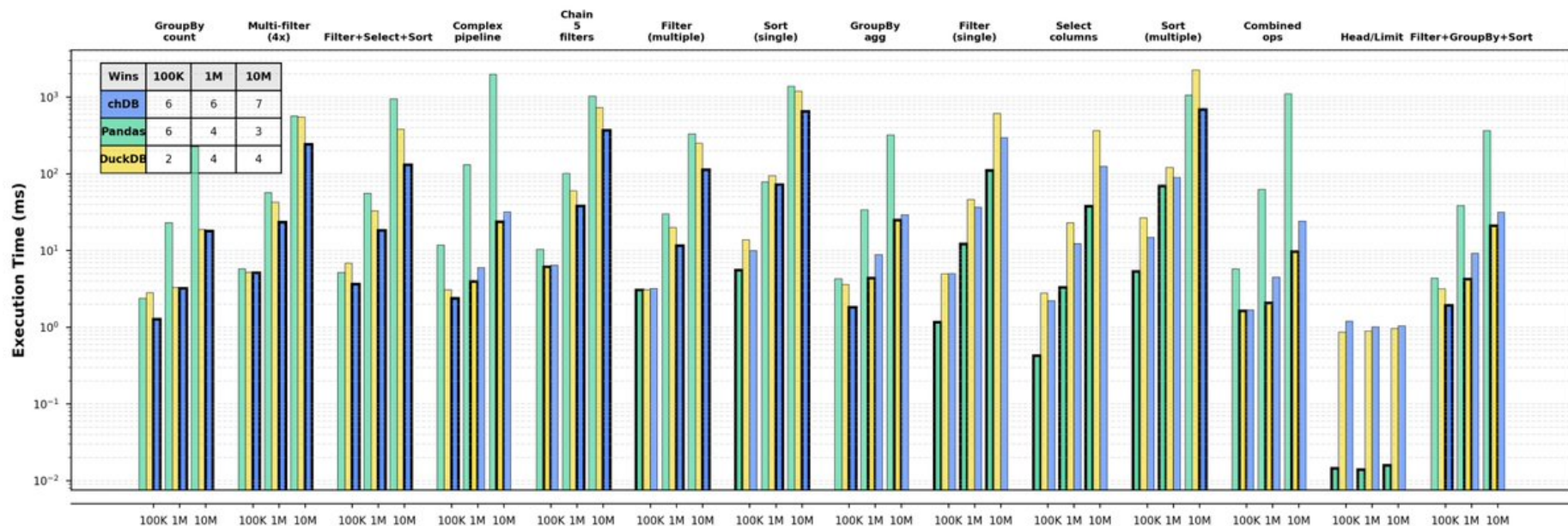


libchdb.so

chDB 够快吗?

DataFrame 基准测试

DataFrame Performance Benchmark: Pandas vs chDB vs DuckDB (All Operations)



chDB 为何那么快?

chDB 为何那么快

因为ClickHouse非常快



chDB 为何那么快

不让Python拖慢速度

- 问题核心 → Python

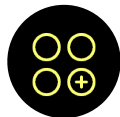


- 解决方案：在 C++ 中实现全并行处理



总结

功能特性和应用场景



简单易用

chDB 按需与业务代码协同运行，无需维护任何高昂的后端服务器架构



极致性能

利用 ClickHouse 的高性能处理能力，chDB 在进程内快速地处理本地或者云端的数据



无缝集成

完整内置 ClickHouse 的 OLAP 功能，深度集成 Python 生态，无需改变查询习惯



快速原型开发

直接在本地的 Jupyter Notebook 中开发，快速验证并展示原型

体验 chDB



文档

chDB 示例与文档: <https://clickhouse.com/docs/chdb>

chDB 起源: <https://auxten.com/the-birth-of-chdb>



问题反馈

提交 Issue: <https://github.com/chdb-io/chdb/issues>

社群讨论: <https://discord.gg/D2Daa2fM5K>



联系方式

邮箱: auxten@clickhouse.com

推特: @auxten

谢谢!